

Cálculo Estocástico para Finanzas

Compendio Completo: Laboratorios, Tarea, Proyecto y Ejercicios

Mauricio Olivares

Profesor: Dr. Julio César Galindo

Universidad Panamericana

Verano 2026

Índice

I	Laboratorios	4
1.	Laboratorio 1: Estimación de un Movimiento Browniano Geométrico	4
1.1.	Marco teórico	4
1.2.	Objetivos	4
1.3.	Código	5
1.4.	Conclusiones	6
2.	Laboratorio 2: Volatilidad Realizada y Ventanas Móviles	7
2.1.	Marco teórico	7
2.2.	Código	7
2.3.	Conclusiones	8
3.	Laboratorio 3: Valuación Monte Carlo de una Opción Europea	8
3.1.	Marco teórico	8
3.2.	Código	9
3.3.	Conclusiones	10
4.	Laboratorio 4: Opción Americana con Longstaff–Schwartz	10
4.1.	Marco teórico	10
4.2.	Código	11
4.3.	Conclusiones	12
5.	Laboratorio 5: Opción Parisina y Reloj de Barrera	13
5.1.	Marco teórico	13

5.2. Código	13
5.3. Conclusiones	15
II Tarea: Procesos de Poisson, Cadenas de Markov y Martingalas Discretas	15
6. Procesos de Poisson	15
7. Cadenas de Markov	16
8. Martingalas Discretas	17
III Proyecto Final: Factores y Estilos de Inversión — AAPL 2019–2024	17
9. Introducción y Pregunta Central	18
10.Marco Teórico	18
10.1. CAPM	18
10.2. Modelo Fama-French 3 Factores (FF3)	18
11.Datos	18
12.Estadísticas Descriptivas	19
13.Resultados	19
13.1. CAPM	19
13.2. Fama-French 3 Factores	19
13.3. Comparación de Modelos	19
14.Conclusiones del Proyecto	20
15.Código del Proyecto	20
IV 20 Ejercicios Resueltos: Browniano e Integral de Itô	21
Ejercicio 1 – Incrementos de Browniano	21
Ejercicio 2 – Ganancia de una estrategia discreta	22
Ejercicio 3 – Isometria de Ito	22
Ejercicio 4 – Variacion cuadratica	22
Ejercicio 5 – Formula de Ito para el logaritmo	22

Ejercicio 6 – Distribucion bajo GBM	23
Ejercicio 7 – Volatilidad realizada	23
Ejercicio 8 – Valuacion de una opcion europea	23
Ejercicio 9 – Put americana vs put europea	23
Ejercicio 10 – Opcion parisina: estado ampliado	23
Ejercicio 11 – Martingala $M_t = B_{t^2} - t$	23
Ejercicio 12 – Martingala exponencial	24
Ejercicio 13 – Precio de mercado del riesgo	24
Ejercicio 14 – Precio descontado como martingala	24
Ejercicio 15 – Paridad put-call	24
Ejercicio 16 – Delta y Gamma	24
Ejercicio 17 – PDE de Black-Scholes	25
Ejercicio 18 – Frontera de ejercicio en put americana	25
Ejercicio 19 – Sensibilidad de opcion parisina	25
Ejercicio 20 – Volatilidad historica vs implicita	25
Bibliografia	26

Parte I

Laboratorios

1. Laboratorio 1: Estimación de un Movimiento Browniano Geométrico

1.1. Marco teórico

El **Movimiento Browniano Geométrico (GBM)** es el modelo estándar para la evolución del precio de un activo financiero. La ecuación diferencial estocástica es:

$$dS_t = \mu S_t dt + \sigma S_t dB_t \quad (1)$$

cuya solución explícita, obtenida mediante la Fórmula de Itô, es:

$$S_t = S_0 \exp \left[\left(\mu - \frac{\sigma^2}{2} \right) t + \sigma B_t \right] \quad (2)$$

Los parámetros se calibran a partir de log-retornos diarios $r_t = \ln(S_t/S_{t-1})$:

$$\hat{\mu} = 252 \cdot \bar{r} + \frac{1}{2}(252 \cdot s^2) \quad (3)$$

$$\hat{\sigma} = \sqrt{252} \cdot s \quad (4)$$

donde s es la desviación estándar muestral de los log-retornos diarios.

1.2. Objetivos

1. Descargar precios ajustados de AAPL (2020–2023).
2. Estimar μ_b y σ_b con log-retornos históricos.
3. Simular 100 trayectorias GBM y comparar con el precio histórico real.
4. Graficar bandas de percentiles (5°, 50°, 95°).

1.3. Código

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 try:
6     import yfinance as yf
7 except ImportError:
8     import os; os.system('pip install yfinance'); import
        yfinance as yf
9
10 ticker_symbol = 'AAPL'
11 df = yf.download(ticker_symbol, start='2020-01-01',
12                 end='2023-12-31', progress=False)
13 if isinstance(df.columns, pd.MultiIndex):
14     df.columns = df.columns.droplevel(1)
15 price_col = 'Adj Close' if 'Adj Close' in df.columns else 'Close'
16 df[price_col] = pd.to_numeric(df[price_col], errors='coerce')
17 df = df.dropna(subset=[price_col])
18
19 df['Log_Return'] = np.log(df[price_col]).diff()
20 df = df.dropna(subset=['Log_Return'])
21
22 mb = df['Log_Return'].mean()
23 sb = df['Log_Return'].std()
24
25 N      = 252
26 mu_b   = N * mb + 0.5 * (N * sb**2)
27 sigma_b = np.sqrt(N) * sb
28 print(f"Mu_b      = {mu_b:.6f}")
29 print(f"Sigma_b   = {sigma_b:.6f}")
30
31 S0      = df[price_col].iloc[-1]
32 num_steps = len(df)
33 num_simulations = 100
34 dt       = 1.0
35 daily_drift = (mu_b - 0.5 * sigma_b**2) / N
36 daily_diffusion = sigma_b / np.sqrt(N)
37
38 simulated_paths = np.zeros((num_steps, num_simulations))
39 simulated_paths[0] = S0
40 np.random.seed(42)
41 for t in range(1, num_steps):
42     Z = np.random.normal(0, 1, num_simulations)
43     simulated_paths[t] = (simulated_paths[t-1]
44                         * np.exp(daily_drift*dt + daily_diffusion*np.sqrt(dt)*Z))
45

```

```

46 lower = np.percentile(simulated_paths, 5, axis=1)
47 med    = np.percentile(simulated_paths, 50, axis=1)
48 upper  = np.percentile(simulated_paths, 95, axis=1)
49
50 fig, ax = plt.subplots(figsize=(14, 6))
51 ax.plot(df.index, df[price_col], color='blue', lw=2,
52         label='Historico real (AAPL)')
53 for i in range(min(20, num_simulations)):
54     ax.plot(df.index, simulated_paths[:, i], color='gray',
55            alpha=0.15)
56 ax.plot(df.index, med, color='red', ls='--',
57         label='Percentil 50')
58 ax.plot(df.index, lower, color='green', ls=':', label='Bandas
59         5-95')
60 ax.plot(df.index, upper, color='green', ls=':')
61 ax.set_title('AAPL vs. Simulaciones GBM', fontsize=14)
62 ax.set_xlabel('Fecha'); ax.set_ylabel('Precio (USD)')
63 ax.legend(); ax.grid(True, ls='--', alpha=0.6)
64 plt.tight_layout(); plt.show()

```

Listing 1: Laboratorio 1 – GBM con AAPL

1.4. Conclusiones

El GBM ofrece una aproximación razonable de la tendencia y rango general de movimiento de precios, pero presenta limitaciones importantes:

1. **Volatilidad no constante (*volatility clustering*):** los mercados exhiben periodos de calma seguidos de estallidos de alta volatilidad, incompatibles con σ constante.
2. **Colas pesadas:** los log-retornos reales tienen colas más gruesas que una normal; el GBM subestima la probabilidad de eventos extremos.
3. **Saltos discontinuos:** noticias y eventos geopolíticos producen saltos que el GBM no puede modelar.
4. **Deriva constante:** el retorno esperado real cambia con el tiempo según fundamentos económicos.

Modelos más avanzados: volatilidad estocástica (Heston), saltos-difusión (Merton), GARCH.

2. Laboratorio 2: Volatilidad Realizada y Ventanas Móviles

2.1. Marco teórico

La volatilidad realizada anualizada con ventana móvil de w días:

$$\hat{\sigma}_t^{(w)} = \sqrt{252} \cdot \text{std}\{r_{t-w+1}, \dots, r_t\} \quad (5)$$

Ventana	Equivalencia	Interpretación
21 días	1 mes bursátil	Alta sensibilidad, mucho ruido
63 días	1 trimestre	Balance sensibilidad/suavizado
126 días	1 semestre	Visión estructural, efecto rezago

Cuadro 1: Ventanas móviles analizadas.

Volatility Clustering (Mandelbrot; Engle, Premio Nobel – modelos ARCH): grandes cambios en precios son seguidos por grandes cambios, y pequeños por pequeños.

2.2. Código

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5
6 try:
7     import yfinance as yf
8 except ImportError:
9     import os; os.system('pip install yfinance'); import
        yfinance as yf
10
11 sns.set_theme(style='whitegrid')
12
13 ticker = 'SPY'
14 datos = yf.download(ticker, start='2014-01-01',
15                     end='2024-01-01', progress=False)
16 if isinstance(datos.columns, pd.MultiIndex):
17     datos.columns = datos.columns.droplevel(1)
18
19 precios = datos['Close'].copy()
20 log_ret = np.log(precios / precios.shift(1)).dropna()
21

```

```

22 ventanas = [21, 63, 126]
23 df_vol = pd.DataFrame(index=log_ret.index)
24 for v in ventanas:
25     df_vol[f'Vol_{v}d'] = log_ret.rolling(window=v).std() *
        np.sqrt(252)
26
27 fig, ax = plt.subplots(figsize=(15, 7))
28 ax.plot(df_vol['Vol_21d'] * 100, label='21d (corto)',
29         color='#e74c3c', alpha=0.6, lw=1.2)
30 ax.plot(df_vol['Vol_63d'] * 100, label='63d (medio)',
31         color='#3498db', alpha=0.8, lw=1.8)
32 ax.plot(df_vol['Vol_126d'] * 100, label='126d (largo)',
33         color='#2c3e50', alpha=0.9, lw=2.5)
34 ax.axvspan(pd.to_datetime('2020-03-01'),
35            pd.to_datetime('2020-06-01'),
36            color='red', alpha=0.12)
37 ax.axvspan(pd.to_datetime('2022-01-01'),
38            pd.to_datetime('2022-12-31'),
39            color='orange', alpha=0.10)
40 ax.set_title(f'Volatilidad Realizada Anualizada -- {ticker}',
41            fontsize=14)
42 ax.set_xlabel('Fecha'); ax.set_ylabel('Volatilidad (%)')
43 ax.legend(); ax.grid(True, ls='--', alpha=0.6)
44 plt.tight_layout(); plt.show()

```

Listing 2: Laboratorio 2 – Volatilidad realizada

2.3. Conclusiones

1. **Regímenes de mercado:** COVID-19 (mar–may 2020) llevó la ventana de 21 días por encima del 60 %; el ciclo de alzas de la Fed (2022) estableció una meseta sostenida del 25–30 %.
2. **Volatility clustering validado:** los picos forman bloques temporales extendidos, no eventos aislados.
3. El supuesto de σ constante del GBM básico queda invalidado empíricamente.

3. Laboratorio 3: Valuación Monte Carlo de una Opción Europea

3.1. Marco teórico

Bajo la medida neutral al riesgo $\mathbb{Q}$:

$$C_0 = e^{-rT} \mathbb{E}^{\mathbb{Q}}[(S_T - K)^+] \quad (6)$$

El método Monte Carlo simula N precios terminales:

$$S_T^{(i)} = S_0 \exp \left[\left(r - \frac{\sigma^2}{2} \right) T + \sigma \sqrt{T} Z^{(i)} \right], \quad Z^{(i)} \sim \mathcal{N}(0, 1) \quad (7)$$

El estimador y su error estándar:

$$\hat{C}_0 = \frac{e^{-rT}}{N} \sum_{i=1}^N (S_T^{(i)} - K)^+, \quad \text{SE} = \frac{e^{-rT} s}{\sqrt{N}} \quad (8)$$

El benchmark analítico es **Black-Scholes**:

$$C_0^{\text{BS}} = S_0 \mathcal{N}(d_1) - K e^{-rT} \mathcal{N}(d_2) \quad (9)$$

$$d_1 = \frac{\ln(S_0/K) + (r + \sigma^2/2)T}{\sigma \sqrt{T}}, \quad d_2 = d_1 - \sigma \sqrt{T} \quad (10)$$

3.2. Código

```

1 import numpy as np
2 import pandas as pd
3 from scipy.stats import norm
4 import datetime
5
6 try:
7     import yfinance as yf
8 except ImportError:
9     import os; os.system('pip install yfinance'); import
        yfinance as yf
10
11 ticker = 'SPY'
12 start =
        (datetime.date.today() - datetime.timedelta(days=365*2)).strftime('%Y-%m-%d')
13 end = datetime.date.today().strftime('%Y-%m-%d')
14 data = yf.download(ticker, start=start, end=end,
        progress=False)
15 if isinstance(data.columns, pd.MultiIndex):
16     data.columns = data.columns.droplevel(1)
17 S0 = data['Close'].iloc[-1].item()
18 lr = np.log(data['Close']/data['Close'].shift(1)).dropna()
19 sigma_b = lr.std().item() * np.sqrt(252)

```

```

20
21 K          = S0 * 1.05
22 T          = 1.0
23 r          = 0.03
24 num_simulations = 100000
25
26 Z          = np.random.standard_normal(num_simulations)
27 ST          = S0 * np.exp((r - 0.5*sigma_b**2)*T +
    sigma_b*np.sqrt(T)*Z)
28 payoffs    = np.maximum(ST - K, 0)
29
30 mc_price    = np.exp(-r*T) * np.mean(payoffs)
31 se          = np.std(payoffs)/np.sqrt(num_simulations) *
    np.exp(-r*T)
32 ci_low     = mc_price - 1.96*se
33 ci_high    = mc_price + 1.96*se
34
35 d1 = (np.log(S0/K)+(r+0.5*sigma_b**2)*T)/(sigma_b*np.sqrt(T))
36 d2 = d1 - sigma_b*np.sqrt(T)
37 bs_price = S0*norm.cdf(d1) - K*np.exp(-r*T)*norm.cdf(d2)
38
39 print(f"MC Price : {mc_price:.4f} SE: {se:.6f}")
40 print(f"IC 95%   : [{ci_low:.4f}, {ci_high:.4f}]")
41 print(f"BS Price : {bs_price:.4f} Diff:
    {mc_price-bs_price:.6f}")

```

Listing 3: Laboratorio 3 – Monte Carlo europeo

3.3. Conclusiones

- El precio MC converge al precio Black-Scholes; la diferencia tiende a cero con $N \rightarrow \infty$ (convergencia del TCL).
- El error estándar decrece en proporción a $1/\sqrt{N}$.
- Monte Carlo es valioso para opciones exóticas sin solución analítica cerrada.

4. Laboratorio 4: Opción Americana con Longstaff–Schwartz

4.1. Marco teórico

Una opción americana satisface el problema de **paro óptimo**:

$$V^{Am}(t, s) = \sup_{\tau \in \mathcal{T}_{t,T}} \mathbb{E}^{\mathbb{Q}} \left[e^{-r(\tau-t)} g(S_{\tau}) \mid S_t = s \right] \quad (11)$$

Como la europea corresponde a $\tau = T$, se tiene $V^{Am} \geq V^{Eu}$. La desigualdad puede ser estricta para puts: ejercer anticipadamente recibe K reinvertible a tasa r .

Algoritmo de Longstaff–Schwartz (LSM): regresión hacia atrás desde T . En cada paso se estima el valor de continuación mediante regresión polinomial sobre las trayectorias in-the-money:

$$\hat{C}(S_t) = \sum_k \alpha_k L_k(S_t) \quad (12)$$

Se ejerce cuando $g(S_t) > \hat{C}(S_t)$.

4.2. Código

```

1 import numpy as np
2 import pandas as pd
3 from scipy.stats import norm
4
5 try:
6     import yfinance as yf
7 except ImportError:
8     import os; os.system('pip install yfinance'); import
        yfinance as yf
9
10 TICKER = 'SPY'
11 data = yf.download(TICKER, start='2023-01-01',
12                     end='2024-01-01', progress=False)
13 if isinstance(data.columns, pd.MultiIndex):
14     data.columns = data.columns.droplevel(1)
15 S0 = data['Close'].iloc[-1].item()
16 lr = np.log(data['Close']/data['Close'].shift(1))
17 sigma_b = lr.std().item() * np.sqrt(252)
18
19 K, T, r = S0*0.95, 1.0, 0.05
20 num_paths = 10000
21 num_steps = 100
22 dt = T / num_steps
23
24 np.random.seed(42)
25 paths = np.zeros((num_steps+1, num_paths))
26 paths[0] = S0
27 for t in range(1, num_steps+1):

```

```

28     Z          = np.random.normal(0, 1, num_paths)
29     paths[t] = paths[t-1] * np.exp(
30         (r - 0.5*sigma_b**2)*dt + sigma_b*np.sqrt(dt)*Z)
31
32     cash_flow      = np.maximum(K - paths[-1], 0)
33     exercise_time = np.full(num_paths, num_steps)
34
35     for t in range(num_steps-1, 0, -1):
36         itm      = np.where(K - paths[t] > 0)[0]
37         if len(itm) == 0: continue
38         S_itm = paths[t][itm]
39         y      = cash_flow[itm] * np.exp(-r*dt*(exercise_time[itm]-t))
40         coef   = np.polyfit(S_itm, y, 2)
41         cont   = np.polyval(coef, S_itm)
42         iv     = np.maximum(K - S_itm, 0)
43         ex     = itm[iv > cont]
44         cash_flow[ex]      = iv[iv > cont]
45         exercise_time[ex] = t
46
47     american_price = np.mean(cash_flow * np.exp(-r*dt*exercise_time))
48
49     d1 = (np.log(S0/K)+(r+0.5*sigma_b**2)*T)/(sigma_b*np.sqrt(T))
50     d2 = d1 - sigma_b*np.sqrt(T)
51     european_price = K*np.exp(-r*T)*norm.cdf(-d2) - S0*norm.cdf(-d1)
52
53     print(f"Put Americana (LSM) : {american_price:.4f}")
54     print(f"Put Europea (BS) : {european_price:.4f}")
55     print(f"Prima Americanidad :
56         {american_price-european_price:.4f}")

```

Listing 4: Laboratorio 4 – Longstaff-Schwartz

4.3. Conclusiones

- La **prima de americanidad** cuantifica el valor de la flexibilidad de ejercicio temprano.
- Para calls sin dividendos la prima es cero (óptimo no ejercer antes).
- LSM converge con $N \geq 10,000$ trayectorias y grado polinomial ≥ 2 ; es el estándar en la industria.

5. Laboratorio 5: Opción Parisina y Reloj de Barrera

5.1. Marco teórico

Una **opción parisina down-and-out** se desactiva si el precio permanece *consecutivamente* por debajo de la barrera H durante D días.

El espacio de estados requiere el reloj de barrera C_t :

$$V = V(t, S_t, C_t) \quad (13)$$

La EDP incluye el término $\partial V / \partial C$ cuando $S_t < H$:

$$V_t + rS V_S + \frac{1}{2} \sigma^2 S^2 V_{SS} + \mathbf{1}_{\{S < H\}} V_C - rV = 0 \quad (14)$$

Sensibilidades:

- $D \uparrow \Rightarrow$ menor prob. de knock-out $\Rightarrow$ precio sube (converge al europeo puro cuando $D \rightarrow \infty$).
- $H \uparrow \Rightarrow$ mayor prob. de knock-out $\Rightarrow$ precio baja.

5.2. Código

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 try:
6     import yfinance as yf
7 except ImportError:
8     import os; os.system('pip install yfinance'); import
        yfinance as yf
9
10 ticker = 'SPY'
11 data = yf.download(ticker, start='2020-01-01',
12                    end='2023-01-01', progress=False)
13 if isinstance(data.columns, pd.MultiIndex):
14     data.columns = data.columns.droplevel(1)
15 S0 = data['Close'].iloc[-1].item()
16 lr = np.log(data['Close']/data['Close'].shift(1)).dropna()
17 sigma_b = lr.std().item() * np.sqrt(252)
18
19 K, T, r = S0*0.95, 1.0, 0.02
20 num_simulations = 100000

```

```

21 num_steps      = 252
22 dt             = T / num_steps
23 H              = S0 * 0.90
24 D_values       = [5, 10, 20, 30]
25
26 np.random.seed(42)
27 paths          = np.zeros((num_simulations, num_steps+1))
28 paths[:, 0]    = S0
29 Z_all          = np.random.normal(0, 1, (num_simulations,
    num_steps))
30 for t in range(num_steps):
31     paths[:, t+1] = paths[:, t] * np.exp(
32         (r - 0.5*sigma_b**2)*dt + sigma_b*np.sqrt(dt)*Z_all[:,
    t])
33
34 def parisian_price(K, T, r, H, D_val, paths):
35     n_sim      = paths.shape[0]
36     payoffs    = np.zeros(n_sim)
37     for i in range(n_sim):
38         path    = paths[i]
39         ko      = False
40         consec  = 0
41         for t in range(1, len(path)):
42             if path[t] < H:
43                 consec += 1
44                 if consec >= D_val: ko = True; break
45             else:
46                 consec = 0
47         if not ko:
48             payoffs[i] = max(0, K - path[-1])
49     price = np.mean(payoffs) * np.exp(-r*T)
50     se    = np.std(payoffs)/np.sqrt(n_sim) * np.exp(-r*T)
51     return price, se
52
53 prices, errors = [], []
54 for D in D_values:
55     p, e = parisian_price(K, T, r, H, D, paths)
56     prices.append(p); errors.append(e)
57     print(f"D={D:2d} dias -> Precio: {p:.4f} SE: {e:.4f}")
58
59 fig, ax = plt.subplots(figsize=(9, 5))
60 ax.errorbar(D_values, prices, yerr=errors, fmt='-o', capsize=5,
61     color='steelblue', ecolor='lightcoral',
62     markerfacecolor='navy')
63 ax.set_title('Precio Parisino vs. Duracion D del Reloj',
64     fontsize=13)
65 ax.set_xlabel('D (dias consecutivos bajo H)')
66 ax.set_ylabel('Precio de la Opcion')

```

```

65 ax.grid(True, ls='--', alpha=0.7)
66 plt.tight_layout(); plt.show()

```

Listing 5: Laboratorio 5 – Opcion parisina

5.3. Conclusiones

- La *memoria temporal* distingue a la opción parisina de la barrera estándar.
- El espacio de estados debe ampliarse a (t, S_t, C_t) ; sin C_t , dos estados financieramente distintos serían indistinguibles.
- Las opciones parisinas modelan escenarios donde la activación requiere confirmación sostenida (p.ej., regulaciones con umbral mantenido varios días).

Parte II

Tarea: Procesos de Poisson, Cadenas de Markov y Martingalas Discretas

6. Procesos de Poisson

Definición 6.1. Un *proceso de Poisson* $\{N(t)\}_{t \geq 0}$ con tasa $\lambda > 0$ satisface:

1. $N(0) = 0$.
2. Incrementos independientes y estacionarios.
3. $P(N(t) = k) = \frac{(\lambda t)^k e^{-\lambda t}}{k!}$, $E[N(t)] = \lambda t$, $\text{Var}[N(t)] = \lambda t$.

Proposición 6.1 (Ley condicional). Para $0 < s < t$: $N(s) | N(t) = n \sim \text{Binomial}(n, s/t)$.

Ejercicio 1.1 — Llegadas de órdenes en una mesa de FX ($\lambda = 10$ órd/hora)

(a) $P(N(0,5) = 0) = e^{-10 \times 0,5} = e^{-5} \approx 0,0067$.

Implicación: periodos de iliquidez donde el riesgo de inventario aumenta.

(b) Por independencia de incrementos en intervalos disjuntos:

$$P(N_{10:30-11} = 3) \cdot P(N_{11:30-12} = 7) = \frac{e^{-5} 5^3}{3!} \cdot \frac{e^{-5} 5^7}{7!} \approx 0,0146$$

(c) $E[N(8)] = 80$, $SD = \sqrt{80} \approx 8,94$.

No es prudente dimensionar solo con la media por la alta volatilidad del flujo.

(d) $SD(N(T)) = \sqrt{\lambda T} \propto \sqrt{T}$, relacionado con la **regla de la raíz del tiempo** del VaR:
 $VaR_{10d} \approx \sqrt{10} VaR_{1d}$.

Ejercicio 1.4 — Ley condicional

$$N(s) | N(t) = n \sim \text{Binomial}\left(n, \frac{s}{t}\right)$$

$$E[N(s) | N(t) = n] = n \frac{s}{t}, \quad \text{Var}[N(s) | N(t) = n] = n \frac{s}{t} \left(1 - \frac{s}{t}\right)$$

Ejercicio 1.8 — Poisson Compuesto (Cramér-Lundberg)

$$E[X(t)] = \lambda t E[Y], \quad \text{Var}[X(t)] = \lambda t E[Y^2].$$

Con $\lambda = 3$ siniestros/mes y $E[Y] = 10,000$ MXN:

$$E[X(12)] = 3 \times 12 \times 10,000 = 360,000 \text{ MXN}$$

7. Cadenas de Markov

Definición 7.1. Una *cadena de Markov* satisface $P(X_{n+1} = j | X_n = i_n, \dots, X_0 = i_0) = p_{ij}$. La **distribución estacionaria** π cumple $\pi P = \pi$, $\sum_i \pi_i = 1$.

Ejercicio 2.1 — Clasificación y distribución estacionaria

Para una matriz de transición P con estados $\{1, 2, 3\}$ (ratings crediticios), se resuelve $\pi P = \pi$ para obtener la distribución de largo plazo.

Ejercicio 2.4 — Absorción y riesgo crediticio

Se resuelve $u_i = \sum_j p_{ij} u_j$ para calcular la probabilidad de absorción en el estado de default (R_2) o solvencia (R_1).

Ejercicio 2.6 — Rating con Default (estado absorbente)

$P^3(1, 4)$ da la probabilidad de default a 3 años para un emisor con rating inicial AAA.

8. Martingalas Discretas

Definición 8.1. Una sucesión $\{M_n\}$ es ***martingala*** respecto a $\{\mathcal{F}_n\}$ si M_n es $\mathcal{F}_n$ -medible, $E[|M_n|] < \infty$ y $E[M_{n+1} | \mathcal{F}_n] = M_n$ c.s.

Ejercicio 3.1 — Modelo CRR

Bajo la medida de riesgo neutro:

$$q = \frac{1 + r - d}{u - d}$$

El precio descontado $\tilde{S}_n = S_n / (1 + r)^n$ es martingala bajo $\mathbb{Q}$, garantizando la ausencia de arbitraje.

Ejercicio 3.3 — Desigualdad de Doob

$$P(M^* \geq \lambda) \leq \frac{E[M_N]}{\lambda}$$

usando $\tau = \min\{k : M_k \geq \lambda\}$.

Ejercicio 3.5 — Criterio de Kelly

Se maximiza $G(f) = E[\log(W_n/W_0)]$. La solución f^* optimiza el crecimiento de largo plazo. Sobreinvertir ($f > f_{\max}$) lleva a $W_n \rightarrow 0$ c.s.

Ejercicio 3.8 — FTAP y Valuación

- **Primer Teorema Fundamental:** no hay arbitraje $\iff$ existe una medida de martingala equivalente (EMM).
- **Segundo Teorema:** completitud del mercado $\iff$ unicidad de la EMM.

Parte III

Proyecto Final: Factores y Estilos de Inversión — AAPL 2019–2024

9. Introducción y Pregunta Central

¿Puede describirse el comportamiento de AAPL mediante exposiciones a estilos o factores empíricos?

10. Marco Teórico

10.1. CAPM

$$E(R_i) = R_f + \beta_i [E(R_m) - R_f] \quad (15)$$

α captura el retorno no explicado; en equilibrio debería ser cero.

10.2. Modelo Fama-French 3 Factores (FF3)

$$R_i - R_f = \alpha + b_1(\text{Mkt-RF}) + b_2 \text{SMB} + b_3 \text{HML} + \varepsilon_i \quad (16)$$

- **SMB** (Small Minus Big): prima de tamaño.
- **HML** (High Minus Low): prima de valor. Carga negativa $\Rightarrow$ perfil *growth*.

11. Datos

- Activo: AAPL (NASDAQ), febrero 2019 – diciembre 2024, **71 observaciones mensuales**.
- Factores FF3 y R_f : Kenneth French Data Library.
- Estimación: OLS con errores robustos HC3 (White, 1980).

12. Estadísticas Descriptivas

Variable	Media (%)	Desv. Est.	Mín. (%)	Máx. (%)	Sharpe
AAPL	2.210	8.398	-14,71	21.35	0.263
Mkt-RF	1.464	5.230	-13,37	13.66	0.280
SMB	-0,291	2.721	-7,10	7.47	—
HML	0.075	3.836	-9,89	9.81	—
RF	0.202	0.175	0.01	0.47	—

Cuadro 2: Estadísticas descriptivas – retornos mensuales (%).

13. Resultados

13.1. CAPM

$$\widehat{AAPL}_{exc} = 0,0003 + 1,4192 \cdot (\text{Mkt-RF}) + \varepsilon \quad R^2 = 76,4\% \quad (17)$$

$\beta = 1,4192$ ($p < 0,01$): por cada 1 % de retorno del mercado, AAPL rinde $\approx 1,42$ % adicional.
 α no significativo ($p = 0,953$).

13.2. Fama-French 3 Factores

$$\widehat{AAPL}_{exc} = -0,0004 + 1,4594 (\text{Mkt-RF}) - 0,0022 \text{ SMB} - 0,0029 \text{ HML} + \varepsilon \quad R^2 = 78,2\% \quad (18)$$

13.3. Comparación de Modelos

Métrica	CAPM	FF3	Δ
R^2	76.38 %	78.15 %	+1.77 pp
AIC	-450,40	-457,10	FF3 mejor
BIC	-445,88	-448,05	FF3 mejor
Test F incremental	—	$F = 4,155$	$p = 0,0198$

Cuadro 3: CAPM vs. Fama-French 3F.

14. Conclusiones del Proyecto

1. El comportamiento de AAPL se describe satisfactoriamente mediante exposiciones a factores sistemáticos (FF3 explica el 78.2 % de la varianza mensual).
2. Factor de mercado dominante: $b_1 \approx 1,46$, $p < 0,01$.
3. Perfil factorial: **large-cap** ($b_2 < 0$), **growth** ($b_3 < 0$), **alta beta** — coherente con mega-capitalización tecnológica (\$3B USD).
4. FF3 mejora al CAPM en R^2 , AIC y BIC; el test F incremental confirma la mejora ($p = 0,020$).
5. α no significativo en ningún modelo $\Rightarrow$ consistente con mercados eficientes.

15. Código del Proyecto

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 try:
6     import yfinance as yf
7 except ImportError:
8     import os; os.system('pip install yfinance'); import
        yfinance as yf
9
10 try:
11     from statsmodels.regression.linear_model import OLS
12     from statsmodels.tools import add_constant
13     import statsmodels.api as sm
14 except ImportError:
15     import os; os.system('pip install statsmodels')
16     from statsmodels.regression.linear_model import OLS
17     from statsmodels.tools import add_constant
18     import statsmodels.api as sm
19
20 try:
21     import pandas_datareader.data as web
22 except ImportError:
23     import os; os.system('pip install pandas-datareader')
24     import pandas_datareader.data as web
25
26 aapl = yf.download('AAPL', start='2019-02-01',
27                    end='2024-12-31', progress=False)
28 if isinstance(aapl.columns, pd.MultiIndex):
```

```

29     aapl.columns = aapl.columns.droplevel(1)
30 aapl_monthly = aapl['Close'].resample('ME').last()
31 aapl_ret      = aapl_monthly.pct_change().dropna()
32 aapl_ret.index = aapl_ret.index.to_period('M')
33
34 ff3 = web.DataReader('F-F_Research_Data_Factors',
35                     'famafrench', start='2019-01',
36                     end='2024-12')[0] / 100
37
38 df = pd.concat([aapl_ret.rename('AAPL'), ff3], axis=1).dropna()
39 df['AAPL_exc'] = df['AAPL'] - df['RF']
40
41 X_capm = add_constant(df['Mkt-RF'])
42 res_capm = OLS(df['AAPL_exc'], X_capm).fit(cov_type='HC3')
43 print("=== CAPM ===")
44 print(res_capm.summary2())
45
46 X_ff3 = add_constant(df[['Mkt-RF', 'SMB', 'HML']])
47 res_ff3 = OLS(df['AAPL_exc'], X_ff3).fit(cov_type='HC3')
48 print("=== Fama-French 3F ===")
49 print(res_ff3.summary2())
50
51 print(f"R2 CAPM: {res_capm.rsquared:.4f} | R2 FF3:
52       {res_ff3.rsquared:.4f}")
53 print(f"AIC CAPM: {res_capm.aic:.2f} | AIC FF3:
54       {res_ff3.aic:.2f}")

```

Listing 6: Proyecto Final – CAPM y FF3 para AAPL

Parte IV

20 Ejercicios Resueltos: Browniano e Integral de Itô

Ejercicio 1 — Incrementos de Browniano

Para $0 \leq s < t \leq u < v$, usando $K(a, b) = \min\{a, b\}$:

$$\text{Cov}(B_t - B_s, B_v - B_u) = K(t, v) - K(t, u) - K(s, v) + K(s, u) = t - t - s + s = 0$$

Al ser gaussianos con covarianza nula $\Rightarrow$ independientes. ■

Ejercicio 2 — Ganancia de una estrategia discreta

$H_0 = 1, H_1 = 2, H_2 = -1$:

$$(H \cdot B)_T = 1 \cdot (B_{t_1} - B_0) + 2 \cdot (B_{t_2} - B_{t_1}) + (-1) \cdot (B_T - B_{t_2}) = -B_{t_1} + 3B_{t_2} - B_T$$

Ganancia acumulada; H_i se fija antes del siguiente incremento (no anticipación). ■

Ejercicio 3 — Isometría de Itô

$H_t = a \mathbf{1}_{(0, T/2]} + b \mathbf{1}_{(T/2, T]}$:

$$E \left[\left(\int_0^T H_t dB_t \right)^2 \right] = a^2 \cdot \frac{T}{2} + b^2 \cdot \frac{T}{2} = E \left[\int_0^T H_t^2 dt \right] \quad \checkmark \quad \blacksquare$$

Ejercicio 4 — Variación cuadrática

Con partición uniforme de $[0, T]$ con malla T/n :

$$E[V_n] = T, \quad \text{Var}[V_n] = \frac{2T^2}{n} \rightarrow 0 \quad \Rightarrow \quad V_n \xrightarrow{P} T = [B]_T \quad \blacksquare$$

Ejercicio 5 — Fórmula de Itô para el logaritmo

$f(x) = \log x$:

$$d(\log S_t) = \frac{dS_t}{S_t} + \frac{1}{2} \left(-\frac{1}{S_t^2} \right) (\sigma S_t)^2 dt = \left(\mu - \frac{\sigma^2}{2} \right) dt + \sigma dB_t \quad \blacksquare$$

Ejercicio 6 — Distribución bajo GBM

$$\log \frac{S_t}{S_0} = \left(\mu - \frac{\sigma^2}{2} \right) t + \sigma B_t \sim \mathcal{N} \left(\left(\mu - \frac{\sigma^2}{2} \right) t, \sigma^2 t \right)$$

S_t sigue distribución log-normal. El término $-\sigma^2/2$ es la **corrección de Itô**. ■

Ejercicio 7 — Volatilidad realizada

$$r_t = \log(S_t/S_{t-1}), \quad s = \sqrt{\frac{1}{n-1} \sum (r_t - \bar{r})^2}, \quad \sigma_{\text{anual}} = \sqrt{252} \cdot s \quad \blacksquare$$

Ejercicio 8 — Valuación de una opción europea

$$C_0 = e^{-rT} E^{\mathbb{Q}}[(S_T - K)^+]$$

La delta $\Delta_t = \partial V / \partial S$ es la cobertura. ■

Ejercicio 9 — Put americana vs. put europea

$$V^{Am}(t, s) = \sup_{\tau \in \mathcal{T}_{t,T}} E^{\mathbb{Q}}[e^{-r(\tau-t)} g(S_\tau) | S_t = s] \geq V^{Eu}(t, s) \quad \blacksquare$$

Ejercicio 10 — Opción parisina: estado ampliado

El precio requiere $V = V(t, S_t, C_t)$. Sin C_t , dos estados financieramente distintos serían indistinguibles. ■

Ejercicio 11 — Martingala $M_t = B_t^2 - t$

$$E[M_t | \mathcal{F}_s] = B_s^2 + (t - s) - t = B_s^2 - s = M_s \quad \checkmark \quad \blacksquare$$

Ejercicio 12 — Martingala exponencial

$$Z_t = e^{\theta B_t - \theta^2 t/2}.$$

$$E[Z_t | \mathcal{F}_s] = Z_s \cdot e^{\theta^2(t-s)/2} \cdot e^{-\theta^2(t-s)/2} = Z_s \quad \checkmark \quad \blacksquare$$

$Z_T = dQ/dP$ es la densidad de Radon-Nikodym (Teorema de Girsanov).

Ejercicio 13 — Precio de mercado del riesgo

$$\lambda = \frac{\mu - r}{\sigma}$$

Con $B_t^{\mathbb{Q}} = B_t^{\mathbb{P}} + \lambda t$, el drift pasa de μ a r bajo $\mathbb{Q}$. $\blacksquare$

Ejercicio 14 — Precio descontado como martingala

$$d(e^{-rt} S_t) = \sigma S_t^* dB_t^{\mathbb{Q}}$$

Solo término de difusión $\Rightarrow$ drift nulo $\Rightarrow$ **martingala** bajo $\mathbb{Q}$. $\blacksquare$

Ejercicio 15 — Paridad put–call

$$\boxed{C_0 - P_0 = S_0 - Ke^{-rT}} \quad \blacksquare$$

Ejercicio 16 — Delta y Gamma

$$\Delta V \approx \Delta \cdot \Delta S + \frac{1}{2} \Gamma (\Delta S)^2 + \Theta \Delta t$$

$\Delta = \partial V / \partial S$: cobertura lineal. $\Gamma = \partial^2 V / \partial S^2$: curvatura; $\Gamma > 0$ beneficia de la volatilidad. $\blacksquare$

Ejercicio 17 — PDE de Black–Scholes

$$\boxed{V_t + \frac{1}{2}\sigma^2 S^2 V_{SS} + rS V_S - rV = 0, \quad V(T, S) = h(S)} \quad \blacksquare \quad (19)$$

Ejercicio 18 — Frontera de ejercicio en put americana

Condiciones de *smooth pasting* en $S^*(t)$:

$$V(t, S^*(t)) = K - S^*(t), \quad \frac{\partial V}{\partial S}(t, S^*(t)) = -1$$

$S^*(t)$ decreciente; $S^*(T) = K$. $\blacksquare$

Ejercicio 19 — Sensibilidad de opción parisina down-and-out

- $H \uparrow$: mayor prob. de knock-out $\Rightarrow$ **precio baja**.
- $D \uparrow$: menor prob. de desactivación $\Rightarrow$ **precio sube**.

Se invierte para knock-in. $\blacksquare$

Ejercicio 20 — Volatilidad histórica vs. implícita

	Volatilidad histórica	Volatilidad implícita
Dirección	Backward-looking	Forward-looking
Estimación	$\sqrt{252} \cdot \text{std}\{r_t\}$	Se despeja de $C_{\text{mercado}} = BS(\sigma)$

Cuadro 4: Volatilidad histórica vs. implícita.

Su diferencia es la **variance risk premium**. $\blacksquare$

Bibliografía

1. Fama, E.F. & French, K.R. (1993). Common risk factors in the returns on stocks and bonds. *Journal of Financial Economics*, 33(1), 3–56.
2. Longstaff, F.A. & Schwartz, E.S. (2001). Valuing American options by simulation. *Review of Financial Studies*, 14(1), 113–147.
3. Shreve, S.E. (2004). *Stochastic Calculus for Finance II*. Springer.
4. Sharpe, W.F. (1964). Capital asset prices. *Journal of Finance*, 19(3), 425–442.
5. White, H. (1980). A heteroskedasticity-consistent covariance matrix estimator. *Econometrica*, 48(4), 817–838.